```
'''buildmap.py

   This is the skeleton code for building a map.

   PLEASE FINISH WRITING THE CODE, especially where marked by FIXME.

   This explores the occupancy grid map.

   Node:        /buildmap
   Subscribe:   /scan                    sensor_msgs/Lsaerscan
   Publish:     /map                     nav_msgs/OccupancyGrid
'''

import numpy as np

# ROS Imports
import rclpy

from rclpy.node                 import Node
from rclpy.time                 import Time, Duration
from rclpy.executors            import MultiThreadedExecutor
from rclpy.qos                  import QoSProfile, DurabilityPolicy

from tf2_ros                    import TransformException
from tf2_ros.buffer             import Buffer
from tf2_ros.transform_listener import TransformListener

from geometry_msgs.msg          import Point, Quaternion, Pose
from geometry_msgs.msg          import Transform, TransformStamped
from nav_msgs.msg               import OccupancyGrid
from sensor_msgs.msg            import LaserScan


#
#   Global Definitions
#
WIDTH  = 360
HEIGHT = 240

RESOLUTION = 0.05
ORIGIN_X   = -9.00              # Origin = location of lower-left corner
ORIGIN_Y   = -6.00

LFREE     = -0.3         # FIXME.  Set the log odds ratio of detecting freespace
```

```
45 LOCCUPIED = 0.4          # FIXME.  Set the log odds ratio of detecting occupancy
46
47
48 #
49 #   Custom Node Class
50 #
51 class CustomNode(Node):
52     # Initialization.
53     def __init__(self, name):
54         # Initialize the node, naming it as specified
55         super().__init__(name)
56
57         # Create the log-odds-ratio grid.
58         self.logoddsratio = np.zeros((HEIGHT, WIDTH))
59
60         # Create a publisher to send the map data.  Note we use a
61         # quality of service with durability TRANSIENT_LOCAL, so new
62         # subscribers will get the last sent message.  RVIZ and others
63         # expect this for map messages.
64         quality = QoSProfile(durability=DurabilityPolicy.TRANSIENT_LOCAL,
65                              depth=1)
66         self.pub = self.create_publisher(OccupancyGrid, '/map', quality)
67
68         # Instantiate a TF listener. This implicitly fills a local
69         # buffer, so we can quickly retrieve the transform information
70         # we need.  The buffer is filled via incoming TF message
71         # callbacks, so make sure this runs in a seperate thread.
72         self.tfBuffer = Buffer()
73         TransformListener(self.tfBuffer, self, spin_thread=True)
74
75         # Create a subscriber to the laser scan.
76         self.create_subscription(LaserScan, '/scan', self.laserCB, 1)
77
78         # Create a timer for sending the map data.
79         self.timer = self.create_timer(1.0, self.sendMap)
80
81     # Shutdown
82     def shutdown(self):
83         # Destroy the timer and node.
84         self.destroy_timer(self.timer)
85         self.destroy_node()
86
87
88     ######################################################################
89     # Send the map.  Called from the timer at 1Hz.
90     def sendMap(self):
```

```
91          # Convert the log odds ratio into a probability (0...1).
92          # Remember: self.logsoddsratio is a 360x240 NumPy array,
93          # where the values range from -infinity to +infinity.  The
94          # probability should also be a 360x240 NumPy array, but with
95          # values ranging from 0 to 1, being the probability of a wall.
96          # FIXME: Convert the log-odds-ratio into a probability.
97          probability = np.exp(self.logoddsratio) / (1.0 + np.exp(self.logoddsratio))
98
99          # Perpare the message and send.  Note this converts the
100         # probability into percent, sending integers from 0 to 100.
101         now  = self.get_clock().now()
102         data = (100 * probability).astype(int).flatten().tolist()
103
104         self.map = OccupancyGrid()
105         self.map.header.frame_id        = 'map'
106         self.map.header.stamp           = now.to_msg()
107         self.map.info.map_load_time     = now.to_msg()
108         self.map.info.resolution        = RESOLUTION
109         self.map.info.width             = WIDTH
110         self.map.info.height            = HEIGHT
111         self.map.info.origin.position.x = ORIGIN_X
112         self.map.info.origin.position.y = ORIGIN_Y
113         self.map.data                   = data
114
115         self.pub.publish(self.map)
116
117
118     ######################################################################
119     # Utilities:
120     # Set the log odds ratio value
121     def set(self, u, v, value):
122         # Update only if legal.
123         if (u>=0) and (u<WIDTH) and (v>=0) and (v<HEIGHT):
124             self.logoddsratio[v,u] = value
125         else:
126             self.get_logger().warn("Out of bounds (%d, %d)" % (u,v))
127
128     # Adjust the log odds ratio value
129     def adjust(self, u, v, delta):
130         # Update only if legal.
131         if (u>=0) and (u<WIDTH) and (v>=0) and (v<HEIGHT):
132             self.logoddsratio[v,u] += delta
133         else:
134             self.get_logger().warn("Out of bounds (%d, %d)" % (u,v))
135
136     # Return a list of all intermediate (integer) pixel coordinates
```

```
137         # from (start) to (end) pixel coordinates (possibly non-integer).
138         # In classic Python fashion, this excludes the end coordinates.
139         def bresenham(self, start, end):
140             # Extract the coordinates
141             (us, vs) = start
142             (ue, ve) = end
143 
144             # Move along ray (excluding endpoint).
145             if (np.abs(ue-us) >= np.abs(ve-vs)):
146                 return[(u, int(vs + (ve-vs)/(ue-us) * (u+0.5-us)))
147                        for u in range(int(us), int(ue), int(np.sign(ue-us)))]
148             else:
149                 return[(int(us + (ue-us)/(ve-vs) * (v+0.5-vs)), v)
150                        for v in range(int(vs), int(ve), int(np.sign(ve-vs)))]
151 
152 
153         ######################################################################
154         # Laserscan CB.  Process the scans.
155         def laserCB(self, msg):
156             # Grab the transformation between map and laser's scan frames.
157             # Note the below asks for the lastest known transform.  If we
158             # use Time.from_msg(msg.header.stamp) in place of Time(), we
159             # could determine the transform at the exact time of the scan.
160             # But that delay a little if the latest TF data is behind.
161             try:
162                 tfmsg = self.tfBuffer.lookup_transform(
163                     'map', msg.header.frame_id, rclpy.time.Time())
164             except TransformException as ex:
165                 self.get_logger().warn("Unable to get transform: %s" % (ex,))
166                 return
167 
168             # Extract the laser scanner's position and orientation.
169             xc     = tfmsg.transform.translation.x
170             yc     = tfmsg.transform.translation.y
171             thetac = 2 * np.arctan2(tfmsg.transform.rotation.z,
172                                     tfmsg.transform.rotation.w)
173 
174             # Grab the rays: each ray's range and angle relative to the
175             # turtlebot's position and orientation.
176             rmin     = msg.range_min        # Sensor minimum range to be valid
177             rmax     = msg.range_max        # Sensor maximum range to be valid
178             ranges   = msg.ranges           # List of ranges for each angle
179 
180             thetamin = msg.angle_min        # Min angle (0.0)
181             thetamax = msg.angle_max        # Max angle (2pi)
182             thetainc = msg.angle_increment  # Delta between angles (2pi/360)
```

```
183             thetas   = np.arange(thetamin, thetamax, thetainc)
184
185             ##################################################################
186             # FIXME: PROCESS THE LASER SCAN TO UPDATE THE LOG ODDS RATIO!
187             ##################################################################
188
189             # Calculate deltas, scale them based on resolution, set new u,v values
190             dx = xc - ORIGIN_X
191             dy = yc - ORIGIN_Y
192             u0 = int(dx / RESOLUTION)
193             v0 = int(dy / RESOLUTION)
194             us, vs = u0, v0
195             # self.set(u, v, 2.0)
196
197             # Loop through each r, theta combination
198             for r, t in zip(ranges, thetas):
199                 if not np.isfinite(r):
200                     continue
201                 if rmin < r < rmax:
202                     angle = t + thetac
203                     # Find where the ray hit using trigonometry
204                     x = xc + r * np.cos(angle)
205                     y = yc + r * np.sin(angle)
206
207                     dx = x - ORIGIN_X
208                     dy = y - ORIGIN_Y
209
210                     ue = int(dx / RESOLUTION)
211                     ve = int(dy / RESOLUTION)
212
213                     if (ue < 0 or ue >= WIDTH or ve < 0 or ve >= HEIGHT):
214                         continue
215
216                     for (u, v) in self.bresenham((us, vs), (ue, ve)):
217                         self.adjust(u, v, LFREE)
218                     self.adjust(ue, ve, LOCCUPIED)
219
220
221 #
222 #   Main Code
223 #
224 def main(args=None):
225     # Initialize ROS.
226     rclpy.init(args=args)
227
228     # Instantiate the node.
```

```
229         node = CustomNode('buildmap')
230 
231         # Spin, until interrupted.
232         rclpy.spin(node)
233 
234         # Shutdown the node and ROS.
235         node.shutdown()
236         rclpy.shutdown()
237 
238     if __name__ == "__main__":
239         main()
```